Execution-aware anytime replanning for multi-UAV missions: when to stop computing under uncertain execution, with validation on PX4 and Gazebo

Cao-Phuc Ha[a], Anh Nguyen Vu Minh[a], Phu-Nguyen Le[b,*]

[a]*Department of Artificial Intelligence, FPT University, Ho Chi Minh City, Vietnam*
[b]*Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University, 104 Nguyen Van Troi Street, Phu Nhuan Ward, Ho Chi Minh City, 700000, Vietnam*

Abstract

Multi-UAV replanners usually stop their search after a fixed time budget. This paper studies when such a planner should stop computing and publish its plan, with exact and with uncertain execution. AnytimeSwarm repairs the previous plan, improves it with an anytime adaptive large neighbourhood search and stops when a learned prediction of the next improvement no longer exceeds a delay cost derived from the slack of the plan. AnytimeSwarm-R adds a calibrated model of finish-time errors and dissemination latency to this cost. The evaluation has two tiers, a kinematic simulator with wind, speed, service and latency disturbances drawn with common random numbers, and PX4 software-in-the-loop with Gazebo and ROS 2 after a rescaling that preserves every dimensionless ratio. All design choices were frozen before the held-out runs. With exact execution, AnytimeSwarm reached a regret of 0.083% against 0.112% to 0.130% for fixed budgets and stagnation stopping at equal computation. In 34 560 kinematic runs with uncertain execution, search raised the utility from 68.86% with plan repair alone to 72.25% with AnytimeSwarm-R, and AnytimeSwarm-R lost 76% fewer tasks to late plans than a 250 ms budget on a slow planner. It reached the highest mean utility with 15% less computation than AnytimeSwarm, but its advantage over the strongest baselines, at most 0.32 percentage points, was not significant after Holm correction, and it was not confirmed with 40 UAVs or in 120 PX4 flights. Periodic replanning raised the utility of every method by 1.29 to 1.56 points. The kinematic tier predicted the utility of PX4 flights closely ($r = 0.84$) but not differences below one point. Adaptive stopping thus mainly saves computation, and the replanning trigger matters more for the reward served.

Keywords: multi-UAV task allocation, dynamic vehicle routing, anytime planning, metareasoning, replanning, execution uncertainty, PX4, Gazebo, ROS 2

1. Introduction

Teams of unmanned aerial vehicles (UAVs) are used for inspection, delivery and monitoring. During a mission, new tasks may appear, a vehicle may fail, or a deadline may be moved forward. A central planner then repairs the routes of the whole fleet, while some UAVs may already wait for new instructions. Dynamic vehicle routing and multi-robot task allocation offer many methods for this repair [1–3], and metaheuristics such as adaptive large neighbourhood search (ALNS) [4] usually find good plans when enough time is available.

The time given to such a planner is usually fixed in advance, and the same budget is spent on easy and on hard events. This choice may be inefficient for two reasons. Many events leave little room for improvement,

*Corresponding author.
Email addresses: caophucai@gmail.com (Cao-Phuc Ha), anhnvmse203412@fpt.edu.vn (Anh Nguyen Vu Minh), nguyenlp@vaa.edu.vn (Phu-Nguyen Le)

and on the others most of the improvement of an anytime search tends to arrive early. In addition, idle UAVs wait while the planner computes, so a later plan is not always a better plan. Anytime algorithms and metareasoning study this question in general terms [5–10]. In replanning, the cost of time is specific to the current plan. It depends on which UAVs are idle, on the slack in their routes and, in practice, on how reliably the plan will be executed.

Execution uncertainty changes this cost. A planner that assumes exact execution prices the delay of a plan with the slack that the plan promises. When UAVs accelerate, meet wind and take longer than expected at a task, a part of this slack is consumed by flight-time errors that the planner does not see. Errors also accumulate along a route between two replanning calls, which raises the question of when to replan at all. Validation is a further difficulty. Results obtained with a kinematic simulator may not hold for vehicles flown by an autopilot, while a physics-based simulation alone cannot provide the number of missions that a statistical comparison of planners requires.

This paper studies when a multi-UAV replanner should stop computing and publish its plan, first with exact execution and then under execution uncertainty. Its contributions are fourfold. The first is AnytimeSwarm, a replanner that repairs the previous plan, improves it with an anytime ALNS whose search trajectory does not depend on the budget, and stops when a learned prediction of the next improvement no longer exceeds a delay cost derived from the slack of the current plan. A proposition shows that this cost bounds the reward that a later publication loses. The second is AnytimeSwarm-R, an execution-aware stopping rule whose delay cost is the expected reward lost under a calibrated model of the finish-time error and of the expected dissemination latency. It reduces exactly to the deterministic cost when the error model is zero. The third is a two-tier evaluation framework. Tier B is a continuous-time kinematic simulator with acceleration limits, wind, speed and service-time errors and stochastic communication latency, drawn with common random numbers [11] so that every planner faces the same disturbances. Tier C flies the same missions with three PX4 autopilots [12] in Gazebo [13], connected to the planner through ROS 2 [14], after a rescaling that keeps every dimensionless ratio of the decision problem. The fourth is a pre-registered evaluation. All design choices and error models were fixed on development seeds, the configuration was frozen with cryptographic hashes, and the main comparisons were run once on held-out seeds with mission-level paired bootstrap and Holm correction.

The results are mixed, and they are reported as they came out. Under uncertain execution, the adaptive rules served as much reward as the best fixed budget without tuning per fleet scale, and AnytimeSwarm-R did so with less computation than AnytimeSwarm. The pre-registered hypothesis that AnytimeSwarm-R serves more reward than the strongest baselines was not confirmed, neither in the kinematic tier nor in PX4 flights. The replanning trigger had a larger effect on utility than the stopping rule, and the kinematic tier predicted the utility of PX4 flights closely.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 formulates replanning with publication delay and uncertain execution. Section 4 describes AnytimeSwarm and AnytimeSwarm-R. Section 5 presents the evaluation framework and protocol. Section 6 reports the results, Section 7 discusses them together with the limitations of the study, and Section 8 concludes the paper.

2. Related work

Dynamic task allocation and routing. Multi-robot task allocation [1] and dynamic vehicle routing [2, 3] study how tasks can be reassigned when information changes. The planning problem considered here is a prize-collecting routing problem with time windows [15, 16]. Real-time routing has long been limited by the computation available between events. Gendreau et al. parallelised tabu search to fit more optimisation between request arrivals [17], and large neighbourhood search has been adapted to dynamic routing with time windows [18, 19] and to routing with drones [20]. For multi-UAV systems, auction methods such as CBBA [21, 22] reach conflict-free assignments quickly, uncertainty-gated allocation addresses intermittent communication [23], and centralised methods protect valid assignments or repair only affected route segments

when events occur [24, 25]. These studies make the search faster or better within a given budget. The present work asks how much of the budget should be used for each event and how this choice depends on uncertain execution.

Uncertainty in routing and allocation. Stochastic vehicle routing models travel and service times or demands as random variables [26]. Scenario-based approaches plan against sampled futures in dynamic settings [27], and chance-constrained task allocation bounds the probability that a team misses its objectives [28]. These methods change the plan itself. AnytimeSwarm-R keeps the planning model and the search unchanged and uses the uncertainty only to price the delay of publishing a plan. The two ideas are complementary, and a risk-aware planner could be combined with the stopping rule studied here.

Metareasoning and anytime algorithms. Anytime algorithms [5, 8] and the rational allocation of computation [6, 7] form the conceptual basis of this work. Hansen and Zilberstein derive a monitoring policy by dynamic programming over a performance profile and a time-dependent utility [9]. Svegliato et al. predict the performance of an anytime algorithm online [10] and learn meta-level control with model-free reinforcement learning [29], and Sung et al. learn when an anytime motion planner should stop [30]. In these studies, the utility of time is usually a given function. In multi-UAV replanning, it follows from the plan at hand, from the vehicles that wait for it and, as studied here, from how reliably the plan will be executed.

Simulation-based validation of multi-UAV planning. Software-in-the-loop simulation with an autopilot and a physics simulator is a common step between algorithm design and flight tests. PX4 [12] runs the same flight stack in simulation and on hardware, Gazebo [13] provides vehicle dynamics and sensors, and ROS 2 [14] connects planners and vehicles. Such setups are rarely used for statistical comparisons of planners, because each mission runs in real time or close to it. The two-tier design of this paper uses the physical tier to validate a fast simulator that carries the statistical burden.

3. Problem formulation

3.1. Replanning snapshot

A central planner is called whenever a replanning trigger fires (Section 3.4). Each UAV keeps the task that it is currently flying to or serving. This committed task determines when and where the UAV becomes available again and how much energy it has left. Each open task has a location, a release time, a deadline by which its service must be completed, a service duration and a reward. A plan assigns an ordered sequence of open tasks to each UAV, and tasks may remain unassigned. The planning problem is therefore a prize-collecting routing problem with time windows [15, 16]. Arrival, start and finish times along a route are computed in the usual way for time windows [15, 31]. A route is feasible if every service ends before its deadline and the UAV returns to the depot with an energy reserve of 10% of its capacity. Flight consumes energy in proportion to the distance, hovering during service consumes energy over time, and waiting consumes no energy. A plan π is evaluated by

$$J(\pi) = \alpha R(\pi) - \gamma E(\pi) - \delta C(\pi), \tag{1}$$

where R is the reward of the tasks served on time, E is the flown distance in kilometres, and C is the number of tasks that move to a different UAV than in the previous plan. The weights are $\alpha = 1$, $\gamma = 0.05$ per kilometre and $\delta = 0.3$. Plan quality is reported in percent of W, the total reward of the open tasks.

3.2. Publication delay

Computation is not free. A plan that is computed for τ milliseconds is released $\kappa\tau$ milliseconds later in mission time. The latency factor κ converts planning time into mission time. The value $\kappa = 1$ corresponds to a planner that runs in real time on the evaluation computer, whereas larger values represent slower on-board processors. A UAV that is already idle must wait for the new plan, so its whole route starts later, and tasks whose deadline can no longer be met are lost. With $J_{\text{eff}}(\pi, \tau)$ as the objective of π after this shift, the net utility of publishing π after τ milliseconds is

$$U(\pi, \tau) = J_{\text{eff}}(\pi, \tau) - \mu W \tau, \tag{2}$$

where μ is the price of one millisecond of computation as a fraction of W. The planner thus chooses both a plan and the moment at which it stops computing.

3.3. Uncertain execution and dissemination

The planner uses a nominal model: straight legs at the cruise speed of each UAV and nominal service durations. The executed mission differs from this model in two ways. First, the finish time of task j is $F_j = f_j + e_j$, where f_j is the nominal finish time computed at publication and e_j is an execution error caused by acceleration, wind, speed tracking and service-time variability. Second, the plan reaches the vehicles after an additional dissemination latency L that is not under the control of the planner. A task yields its reward only if its actual finish time meets its deadline, which may have been tightened in the meantime. Following common practice in dispatching, a UAV skips a task at departure if the nominal estimate from its actual state already misses the deadline, and the task returns to the pool of open tasks. The performance measure of a mission is its utility, the reward of all tasks served on time divided by the total reward of all tasks of the mission.

3.4. Replanning triggers

Three trigger policies are considered. The event trigger replans on task arrivals, UAV failures and deadline changes only. The periodic trigger additionally replans every T seconds of mission time. The deviation trigger additionally replans when the actual finish of a planned task deviates from its planned finish by more than θ seconds. During planning, every UAV finishes its committed task and then waits for the new plan, and events that arrive during planning are queued. These semantics are identical in both simulation tiers.

4. Method

AnytimeSwarm consists of a repair module, an anytime search and a stopping rule. Fig. 1 illustrates how the three modules interact during one replanning call. AnytimeSwarm-R differs only in the delay cost used by the stopping rule.

4.1. Feasibility-first repair

The previous plan is first restricted to the open tasks. Each route is then walked once, and a task is kept only if the route up to this task is still feasible, so the result is feasible by construction. Tasks of a failed UAV, removed tasks and new tasks are then inserted with parallel regret-2 insertion [4, 32], and only insertions that increase the objective are made. This feasibility-first repair (FFR) yields a publishable plan within milliseconds and provides the starting point of the search.

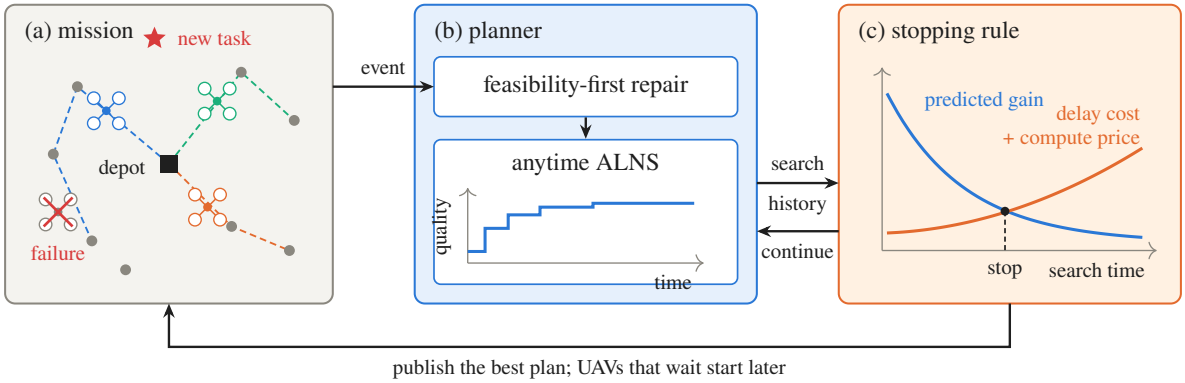


Figure 1: Overview of AnytimeSwarm: (a) an event changes the mission, (b) the planner repairs and improves the plan, (c) the stopping rule decides when to publish it.

4.2. *Anytime adaptive large neighbourhood search*

The plan is then improved by ALNS [4]. Four destroy operators remove between 10% and 35% of the assigned tasks, and at least two. They select tasks at random, by their contribution to the objective, by relatedness [33] or as a segment of a route. Four repair operators apply greedy or regret-2 insertion, each with and without noise. Operators are chosen by adaptive roulette-wheel selection, and new solutions are accepted by simulated annealing [34]. All intermediate solutions are feasible, so the best plan found so far can be published at any time. The temperature decreases per iteration rather than per unit of time, and all variants use the same random seed. Every stopping rule therefore follows the same search trajectory and differs only in the moment at which it stops.

4.3. *Adaptive stopping*

Every 5 ms, the stopping rule compares the predicted gain of searching for another $H = 10$ ms with the cost of doing so. With $\widehat{\Delta J}$ as the predicted gain, the search stops at time τ if

$$\widehat{\Delta J} \;\le\; C_{\text{delay}}(\tau) + \mu W H. \tag{3}$$

The gain is predicted from the search history by a Poisson generalised linear model with a log link [35] and a small L_2 penalty. Its eleven features describe the elapsed time, the number of iterations, the recent improvements over 5 ms, over 20 ms and over the second half of the elapsed time, the time since the last improvement, the total improvement over the repaired plan, the unassigned share of the reward, the numbers of tasks and UAVs, and the iteration rate. The target is the improvement in the next 10 ms in percent of W. The model is fitted on 500 ms search trajectories of development scenarios only, and a prediction costs one dot product. The search loop enforces the hard time budget independently of the stopping rule.

4.4. *Deterministic delay cost*

The delay cost is computed from the best plan found so far. A late start of a route is partly absorbed by the waiting times along it, in the same way as the forward time slack of routing with time windows [31]. The tolerance tol_j of a task is the margin between its finish time and its deadline, plus all waiting times on the route up to this task. A task is lost when the start shift $\sigma_j(\tau)$ of its route exceeds its tolerance. The start shift is zero for a UAV that is still busy, and it equals the waiting time until the publication for an idle UAV. With v_j as the contribution of task j to the objective and $\Lambda(\tau) = \sum_j v_j \mathbb{1}[\sigma_j(\tau) > \text{tol}_j]$, the delay cost of searching H milliseconds longer is $C_{\text{delay}}(\tau) = \Lambda(\tau + H) - \Lambda(\tau)$.

Proposition 1. *For a feasible plan executed exactly, (i) $\Lambda(\tau)$ is the reward lost through missed deadlines if every UAV follows its route, (ii) the loss of the objective is at most $\Lambda(\tau)$ if late tasks are skipped, and (iii) $\Lambda(\tau)$ does not decrease with τ and is zero while all UAVs are busy with their committed tasks. Hence C_{delay} is non-negative and vanishes while no UAV is waiting.*

Proof. A route that starts later by some shift reaches its first task later by the same amount. Each waiting time absorbs part of the delay, so the delay at the end of a task equals the start shift minus all earlier waiting times, if this difference is positive. The task therefore misses its deadline exactly when the start shift exceeds its tolerance. Waiting consumes no energy, which proves (i). If late tasks are skipped, the triangle inequality ensures that every later task is reached no later and with no more energy. All tasks that are on time in (i) remain on time, and the flown distance does not grow. Every assigned task has a positive value, because each reward is at least α and $\delta < \alpha$, which proves (ii). Part (iii) holds because the start shift does not decrease with τ and every tolerance is non-negative in a feasible plan.

4.5. Execution-aware delay cost

Under uncertain execution, task j is lost when $\sigma_j(\tau) + e_j > \text{tol}_j$, where e_j is its finish-time error. AnytimeSwarm-R replaces the indicator in Λ by the probability of this event. The error of task j is modelled as $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$, where $h_j = f_j - t_0$ is its lead time, the nominal finish time minus the time of the planning snapshot. The mean and the spread are linear in the lead time,

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h, \tag{4}$$

because errors accumulate along a route. The expected dissemination latency $\bar{L}$ is added to the start shift. The expected loss and the execution-aware delay cost are

$$\Lambda_{\text{R}}(\tau) = \sum_j v_j \, \Phi\!\left(\frac{\sigma_j(\tau) + \bar{L} + m(h_j) - \text{tol}_j}{s(h_j)}\right), \qquad C_{\text{R}}(\tau) = \Lambda_{\text{R}}(\tau + H) - \Lambda_{\text{R}}(\tau), \tag{5}$$

where Φ is the standard normal distribution function and the start shift now includes $\bar{L}$ for idle UAVs. Search and predictor are unchanged, and C_{R} replaces C_{delay} in Eq. (3).

Proposition 2. *(i) With $b_0 = b_1 = s_0 = s_1 = 0$ and $\bar{L} = 0$, $\Lambda_{\text{R}} = \Lambda$ and AnytimeSwarm-R takes exactly the decisions of AnytimeSwarm. (ii) $\Lambda_{\text{R}}(\tau)$ does not decrease with τ, so $C_{\text{R}} \geq 0$. (iii) Under the error model, $\Lambda_{\text{R}}(\tau)$ is the expected value of the tasks whose deadline is missed if every UAV follows its route.*

Proof. With zero spread, Φ is read as the indicator of a positive argument, which gives (i). The start shift does not decrease with τ and Φ is non-decreasing, which gives (ii). Part (iii) follows from the proof of Proposition 1, because task j misses its deadline exactly when its delay at the finish, $\sigma_j + e_j$ minus the absorbed waiting time, is positive, and the expectation of the indicator of this event is the probability in Eq. (5).

The execution-aware cost differs from the deterministic one in two ways. It grows smoothly before a task becomes infeasible, because a task with little remaining slack is already at risk, and it can be positive for a busy UAV through $\bar{L}$. It is also smaller than the deterministic cost for tasks whose tolerance is already consumed by the expected error, because waiting longer cannot lose a task that is likely lost anyway. The parameters of Eq. (4) are fitted on development missions. The mean is estimated by least squares, and the spread by least squares on the absolute residuals scaled by $\sqrt{\pi/2}$, which is less sensitive to the heavy tail of the errors than squared residuals. The latency $\bar{L}$ is the mean of the latency distribution in tier B and the measured mean plan-acceptance latency in tier C.

Table 1: Scenario generator parameters (distances in m, times in s, energy in metres of flight).

Map side / horizon	10 000 / 1 800	Service time	$U[30, 90]$
UAV speed	$U[10, 16]$ m/s	Time-window slack	$U[180, 600]$
Energy capacity	$U[18, 28]$ km	Release delay	$U[0, 120]$
Energy reserve	10% of capacity	Rewards 1, 2, 3, 5	probabilities .4, .3, .2, .1
Hover cost	6 per second	Hotspots	3, std. 600, 50% of tasks
Tasks known at start	50%	Scales (UAVs/tasks)	5/25, 10/50, 20/140, 40/280

5. Evaluation framework

5.1. Scenarios

Missions are created by a seeded generator with the parameters in Table 1. The map is a square with a side of 10 km and the depot at its centre, and half of the tasks are clustered around three hotspots. Half of the tasks are known at the start, and the others arrive in batches. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles. The arrivals profile contains six arrival batches, the failures profile three batches and two UAV failures, and the deadlines profile three batches and four tightened deadlines. A fourth scale with 40 UAVs and 280 tasks is used for scalability only.

5.2. Static snapshots with exact execution

The stopping rules are first compared without execution uncertainty. Each mission is simulated once with a deterministic reference planner, and every replanning call after the start is stored with its previous plan. All methods are evaluated on these snapshots, and regret is measured against the best known objective of each snapshot, which includes a 3 s ALNS run and, for the small and medium scales, the OR-Tools routing solver [36] with guided local search [37]. Because all ALNS variants follow the same search trajectory, whole families of each rule are evaluated exactly offline, with 12 fixed budgets, 10 stagnation patience values and 9 compute prices, and each baseline is compared with AnytimeSwarm at the same mean computation. This benchmark uses its own pre-registered test seeds 2000–2029 (1562 snapshots of 270 missions), and it was reproduced on the final software platform before the execution-aware study (Section 6.2).

5.3. Tier B: kinematic execution with disturbances and latency

Tier B executes each published plan in continuous mission time. The duration of a leg follows a trapezoidal speed profile with a horizontal acceleration limit. The cruise airspeed of each (UAV, task) leg deviates from the nominal speed by a multiplicative error, and a constant wind of fixed magnitude and random direction changes the ground speed through the wind triangle. Service durations are multiplied by a random factor. Energy is counted as metres flown through the air plus the hover cost. The planner never sees these disturbances. It receives the state of each UAV as telemetry, and the remaining time of a committed task is estimated with the nominal model from the actual position of the UAV. The dissemination latency of a plan is lognormal, and each message is lost with a given probability and resent after a timeout. Table 2 lists the disturbance and latency levels. They span a range from mild to severe conditions and are not claimed to represent a particular vehicle.

All random draws use common random numbers [11]. The speed error of a leg depends only on the mission seed, the UAV and the task, the service factor only on the mission seed and the task, the wind only on the mission seed, and the latency only on the mission seed and the event. Two planners that assign the same task to the same UAV therefore experience the same disturbance, independently of the number of replanning calls. With zero disturbance, zero latency and the event trigger, tier B reproduces the discrete-event simulator of the original formulation exactly, which is verified by unit tests.

Table 2: Disturbance and latency levels of tier B.

Execution	accel. (m/s^2)	speed c.v.	wind (m/s)	service c.v.	Latency	median (s)	$\sigma_{\log}$	loss / timeout
nominal	∞	0	0	0	none	0	–	–
low	3.0	0.03	2	0.05	low	0.2	0.5	–
mid	2.0	0.07	5	0.15	high	2.0	0.8	5% / 5 s
high	1.5	0.12	8	0.25				

5.4. Tier C: PX4 software-in-the-loop with Gazebo and ROS 2

Tier C flies missions with three X500 quadrotors, each controlled by its own PX4 v1.17 autopilot in software-in-the-loop mode [12], in Gazebo Harmonic [13], on ROS 2 Jazzy [14] (Fig. 2). A uXRCE-DDS bridge exposes the internal topics of each autopilot to ROS 2. Three nodes form the mission stack. The mission manager runs the scenario timeline, builds the planning snapshot from the fleet state, calls the planner and releases the plan. The executor streams offboard setpoints to every autopilot at 10 Hz, so that PX4 does not leave offboard mode while the planner computes. It flies each leg with a carrot that moves along the straight leg at the commanded ground speed, holds the UAV at the task for the service duration and reports service start, finish and skips. A visualisation node draws tasks, routes and planning decisions in RViz 2 and as coloured pillars in the Gazebo scene. Planning time is measured with a monotonic wall clock, and every mission time is simulation time.

A tier C mission is a tier B scenario with three UAVs and fifteen tasks, which keeps the ratio of five tasks per UAV of the small scale, rescaled by $S_X = 0.1$ in length and $S_T = 0.25$ in time. Speeds are multiplied by S_X/S_T, energy by S_X, the hover cost by S_X/S_T and the distance price by $1/S_X$. Every dimensionless ratio of the decision problem is preserved, so a tier C mission is the same decision problem as its tier B counterpart, flown in a 1 km world in 450 s with cruise speeds of 4–6.4 m/s. A planning delay of τ in tier C corresponds to τ/S_T in tier B, so the latency factors are related by $\kappa_C = S_T\kappa_B$. Disturbances use the same common random numbers as tier B. The commanded ground speed of each leg follows the speed error and the wind triangle, with the wind scaled by S_X/S_T, and the service duration follows the same factor. Acceleration, attitude dynamics, position tracking and communication are those of the simulated vehicles. Gazebo runs in lockstep with PX4 at twice real time, which requires $\kappa_C \geq 2$ so that the emulated planning delay dominates the real one.

5.5. Compared methods

The constructive baseline is regret-2 repair, that is, FFR without search. ALNS is run with the full budget of 250 ms, with a fixed budget of 40 ms, which equals the mean computation of AnytimeSwarm on development data, and with a stagnation rule that stops after 20% of the budget without improvement. AnytimeSwarm is used exactly as frozen after development, without retraining, and an ablation removes its delay cost. AnytimeSwarm-R uses the error model of the execution level of the mission and the expected latency of its latency level. A further ablation sets its error model to zero, so that only the latency term remains, and a sensitivity study uses the error model of another level. For 40 UAVs, the fixed budget matched to the computation of AnytimeSwarm on development data is 180 ms. All ALNS variants share operators, parameters, initial plan and seed, and every call has a hard budget of 250 ms.

5.6. Protocol and statistics

Development used seeds 0–9 in tier B and 0–2 in tier C. The predictor was trained on development trajectories, and the compute price $\mu = 10^{-5}$ was selected by a rule stated before any test run. The tier B error models were fitted on 90 development missions per level flown by ALNS with 40 ms, and the tier C models on nine development flights per level. The fitted models are listed in Table 3. The experiment configurations, the

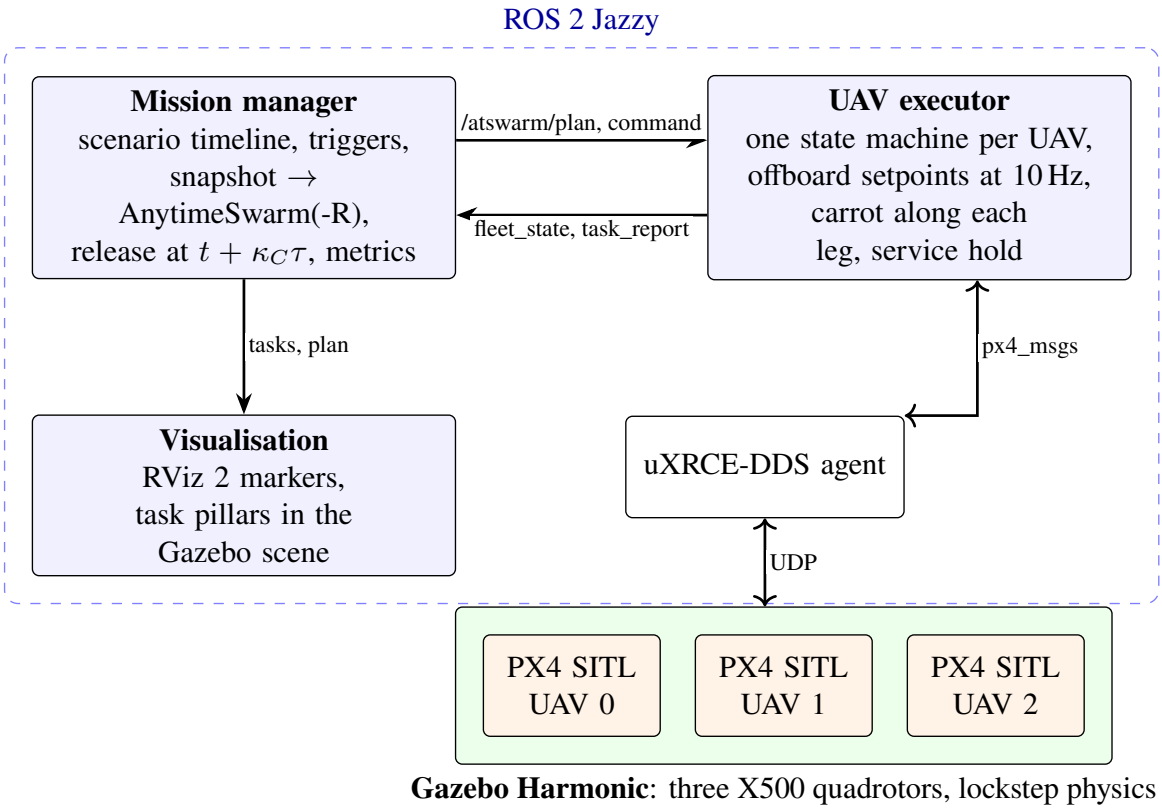


Figure 2: Tier C architecture. The planner and the mission timeline run in the mission manager; the executor streams offboard setpoints to three PX4 autopilots that fly X500 quadrotors in Gazebo.

error models, the predictor and the experiment code were then frozen with SHA-256 hashes, and the held-out runs checked these hashes before starting. Tier B uses held-out seeds 3000–3029, which gives 270 missions for the three scales and three profiles, and 90 missions for 40 UAVs. Tier C uses held-out seeds 4000–4009 with three profiles, that is, 30 missions per method. The primary tier B matrix crosses two execution levels (low, mid), two latency levels (none, high), two latency factors ($\kappa_B = 10, 50$) and two triggers (event, periodic every 60 s).

The primary hypothesis is that AnytimeSwarm-R serves more reward than ALNS with 40 ms, AnytimeSwarm and the stagnation rule over the primary matrix. For each mission, the utility difference is averaged over the sixteen configurations of the matrix, and the mean over missions is tested with a paired bootstrap [38] of 10 000 resamples. The three comparisons are corrected with the Holm procedure [39]. The same pooled test compares the computation per planning call with that of AnytimeSwarm and the periodic trigger with the event trigger. In tier C, AnytimeSwarm-R is compared with ALNS with 40 ms and with AnytimeSwarm, again with Holm correction, and the same 30 missions are also run in tier B to assess whether tier B ranks the methods as tier C does. All other comparisons are descriptive. A tier C run in which a vehicle does not reach its altitude within 90 s of simulation time is aborted and repeated once with the same seed. The full protocol was written before the held-out runs and is available with the code, together with a list of the deviations from it. The only deviation that affects data concerns four tier C runs with a low real-time factor (Section 6.7). Tier B runs four missions in parallel on an Intel Core i7-11800H laptop, with all methods of a configuration run in turn, so that they share the same machine load. All runs use Python 3.12 and pinned package versions. Every plan is checked for feasibility before publication, and a separately written validator

Table 3: Finish-time error models fitted on development missions: $m(h) = b_0 + b_1 h$, $s(h) = s_0 + s_1 h$ (seconds of the respective tier; n executed tasks).

Tier	Level	n	b_0	b_1	s_0	s_1
B	low	5399	8.91	−0.0044	0.00	0.0904
B	mid	4991	16.84	0.0354	1.58	0.1951
B	high	4253	20.79	0.2923	0.00	0.6389
C	low	90	−1.23	0.0281	1.30	0.0545
C	mid	89	2.36	0.0297	0.00	0.2303

Table 4: Regret at equal computation per fleet scale (UAVs/tasks). Baselines run for the mean time of AnytimeSwarm on the same scale; †: 95% interval of the paired difference below zero. Right: computation a baseline needs to match AnytimeSwarm.

		Regret at equal computation (%)			Computation to match (ms)	
Scale	Time (ms)	Proposed	Fixed	Stagnation	Fixed	Stagnation
Small (5/25)	18	0.006	0.030†	0.029†	50 (2.7×)	51 (2.8×)
Medium (10/50)	25	0.030	0.062†	0.045†	49 (2.0×)	39 (1.5×)
Large (20/140)	81	0.213	0.290†	0.274†	172 (2.1×)	135 (1.7×)
40 UAVs (40/280)	151	0.205	0.256†	0.234†	299 (2.0×)	258 (1.7×)
Small to large	42	0.083	0.141†	0.127†	143 (3.4×)	99 (2.4×)

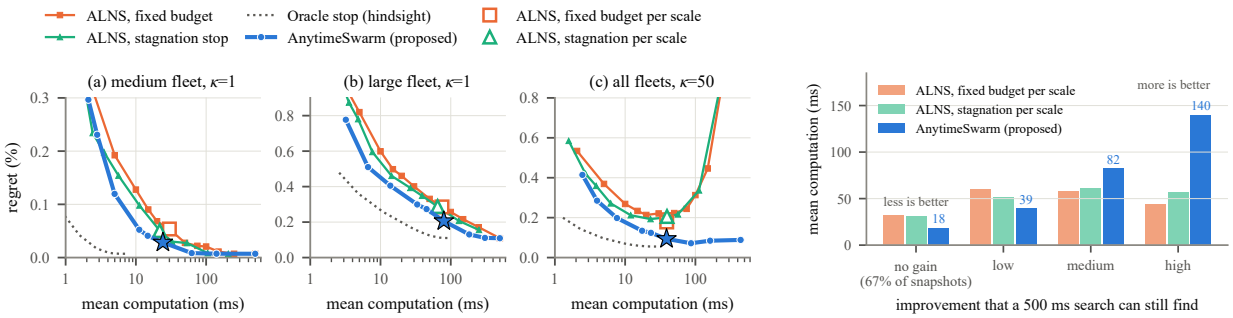
that shares no code with the planner found no infeasible plan among 4 779 development plans of all methods.

6. Results

6.1. Exact execution: stopping rules at equal computation

On the 1562 test snapshots, regret-2 repair finished within about one millisecond but left a regret of 0.98%. With a budget of 250 ms, ALNS with the full budget reached 0.06%, whereas AnytimeSwarm reached 0.08% with 16% of the budget, that is, with 84% less computation. Because the stopping rules use different amounts of computation, they are best compared at equal computation. Table 4 makes this comparison for each fleet scale, with the fixed budget and the stagnation rule evaluated at the mean computation that AnytimeSwarm used on the same scale. AnytimeSwarm reached a lower regret on every scale, and all paired intervals lie below zero. The reduction was 33% to 52% on the medium scale, 22% to 27% on the large scale and 12% to 20% with 40 UAVs, a scale that was never used for training. Read in the other direction, the two baselines needed between 1.5 and 2.8 times the computation of AnytimeSwarm to reach its regret. Over all scales, AnytimeSwarm reached 0.083% against 0.112% to 0.130% for four baselines that include a fixed budget and a stagnation patience tuned per fleet scale on development data.

Fig. 3 shows where this advantage comes from. On the medium and large scales, the quality–compute frontier of AnytimeSwarm lies below those of fixed budgets and stagnation stopping over the range of practical computation (Fig. 3a). Grouped by the improvement that a 500 ms search can still find (Fig. 3b), AnytimeSwarm spent 18.2 ms on snapshots that cannot be improved and 140.0 ms on those with the largest available improvement. Only 29% of its computation went to snapshots without any possible gain, compared with more than half for the baselines. The learned predictor is essential for this allocation. It reached an area under the ROC curve [40] of 0.926 for the event that the next 10 ms bring an improvement, compared with 0.757 for an extrapolation of the recent improvement rate.



(a) (b)

Figure 3: Exact execution, test seeds 2000–2029: (a) quality–compute frontiers on identical search trajectories (open markers: baselines tuned per scale; star: selected AnytimeSwarm setting); (b) mean computation per snapshot, grouped by the improvement that a 500 ms search can still find.

6.2. *Reproduction on the evaluation platform*

The exact-execution benchmark was produced on Windows. Before the execution-aware study, its development pipeline was re-run on the platform of all later experiments (Ubuntu 24.04 under WSL2, identical Python and package versions). The deterministic part was reproduced bit for bit, except for differences of one unit in the last place of some energy values, which stem from the floating-point libraries of the two operating systems. The search ran 21% more iterations per millisecond on the new platform. The improvement predictor, re-trained on the new timing traces, reached the same compute price by the pre-registered rule, and the conclusions of the equal-computation comparison agreed in sign and significance in 24 of 24 comparisons. The frozen predictor was used unchanged in all later experiments.

6.3. *Tier B: stopping rules under uncertain execution*

The primary matrix comprises 34 560 mission runs, that is, 270 held-out missions in 16 configurations with eight methods. Table 5 lists utility, lost tasks and computation per method, and Fig. 4 relates utility to computation and shows the difference in every configuration.

Search pays off clearly under uncertain execution. Regret-2 repair reached a utility of 68.86%, and AnytimeSwarm-R reached 72.25%, which is +3.39 percentage points (pp) or 4.9% more. The full budget of 250 ms was the weakest search method, with 71.48%, and AnytimeSwarm-R served +0.77 [+0.31, +1.25] pp more reward than it. The loss of the long budget stems from late plans. A task counts as lost to delay when it is part of the computed plan but can no longer be served on time when the plan is released. With $\kappa_B = 50$, the full budget lost 5.36 tasks per mission in this way, against 1.28 for AnytimeSwarm-R, that is, 76% fewer. Without its delay cost, AnytimeSwarm lost 3.50 tasks per mission, so the delay cost keeps its role when execution is uncertain.

Among the remaining search methods, the differences are small. All six search methods lie within 0.77 pp of each other. AnytimeSwarm-R reached the highest mean utility with the least computation of the adaptive rules, 41.1 ms per call against 48.2 ms for AnytimeSwarm and 59.9 ms for the stagnation rule. The reduction of 15% relative to AnytimeSwarm is significant (−7.1 [−7.7, −6.5] ms per call), so the second hypothesis holds. The primary hypothesis does not hold at the pre-registered level. The pooled advantage of AnytimeSwarm-R was +0.32 [+0.01, +0.63] pp over ALNS with 40 ms, +0.21 [−0.08, +0.52] pp over AnytimeSwarm and +0.28 [−0.11, +0.67] pp over the stagnation rule, with Holm-adjusted p-values of 0.128, 0.308 and 0.308. Only the first interval excludes zero before correction. Per configuration, AnytimeSwarm-R was significantly better than ALNS with 40 ms in 2 of the 16 configurations, better than the stagnation rule in 1 and never significantly different from AnytimeSwarm. It was significantly worse than none of the baselines in any configuration (Fig. 4b). The ablation that keeps only the latency term reached 72.02%, close to

Table 5: Tier B primary matrix: 270 held-out missions × 16 configurations. Utility and lost tasks per mission, mean planning time per call, and pooled paired difference of AnytimeSwarm-R minus each method (percentage points, 95% bootstrap interval; Holm-adjusted p for the three primary comparisons).

Method	Utility (%)	Late	Skipped	Calls	ms/call	AnytimeSwarm-R minus method (pp)
Regret-2 repair	68.86	6.64	18.35	15.6	1.0	+3.39 [+2.76, +4.04]
ALNS, 250 ms	71.48	6.15	20.58	15.5	250.1	+0.77 [+0.31, +1.25]
ALNS, 40 ms	71.93	6.65	20.97	15.5	40.1	+0.32 [+0.01, +0.63], p_{Holm} 0.128
ALNS, stagnation	71.97	6.48	20.92	15.5	59.9	+0.28 [−0.11, +0.67], p_{Holm} 0.308
AnytimeSwarm w/o delay cost	72.13	6.35	20.49	15.5	54.1	+0.12 [−0.20, +0.45]
AnytimeSwarm	72.04	6.64	21.32	15.5	48.2	+0.21 [−0.08, +0.52], p_{Holm} 0.308
AnytimeSwarm-R, latency only	72.02	6.65	21.31	15.5	48.2	+0.23 [−0.06, +0.54]
AnytimeSwarm-R (proposed)	72.25	6.56	21.15	15.5	41.1	–

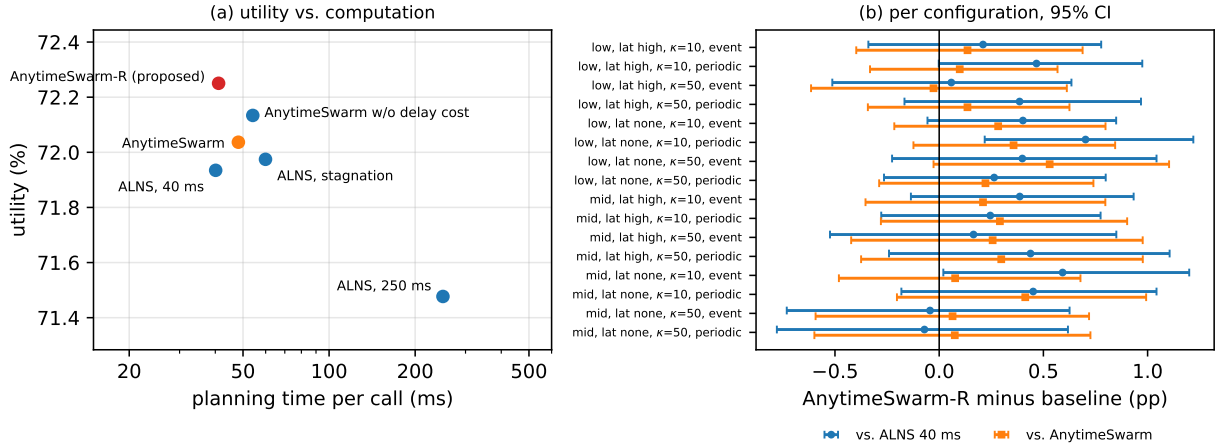


Figure 4: Tier B primary matrix, held-out seeds 3000–3029: (a) mean utility against mean planning time per call over all 16 configurations (regret-2 repair, at 68.86% and 1 ms, is not shown); (b) paired difference of AnytimeSwarm-R minus ALNS with 40 ms and minus AnytimeSwarm per configuration (execution level, latency level, κ_B, trigger) with 95% bootstrap intervals over 270 missions.

AnytimeSwarm, so the small advantage of AnytimeSwarm-R, if real, comes from the error model and not from the expected latency.

The fixed budget of 40 ms is a strong reference, because it was matched to the mean computation of AnytimeSwarm on development data. Its weakness lies in the allocation across fleet scales. AnytimeSwarm-R used 24 ms per call on the small scale and 75 ms on the large scale. On the large scale, it served +0.59 [+0.34, +0.82] pp more reward than the budget of 40 ms and was on a par with the full budget (−0.06 [−0.38, +0.26] pp). On the small scale, it served +1.79 [+0.65, +3.00] pp more than the full budget and was on a par with the budget of 40 ms (+0.33 [−0.40, +1.09] pp). A single fixed budget is therefore either too long for small fleets or too short for large ones, whereas the adaptive rules find a suitable amount of computation on each scale without tuning. The advantage over ALNS with 40 ms was +0.43 [+0.11, +0.76] pp for $\kappa_B = 10$ and +0.20 [−0.19, +0.61] pp for $\kappa_B = 50$, and +0.36 [+0.01, +0.73] pp under low and +0.27 [−0.13, +0.66] pp under mid disturbances.

6.4. Replanning triggers

The trigger mattered more than the stopping rule. Replanning every 60 s in addition to the events raised the number of planning calls from 6.8 to 24.2 per mission and the utility of every method, by +1.29 pp

to +1.56 pp, with all intervals above zero (+1.56 [+1.26, +1.84] pp for AnytimeSwarm-R, +1.47 [+1.19, +1.75] pp for ALNS with 40 ms). The third hypothesis therefore holds. Periodic replanning probably helps because it corrects the accumulated deviations of the executed routes, which the event trigger only notices at the next event. The advantage of AnytimeSwarm-R over ALNS with 40 ms was +0.27 [−0.07, +0.62] pp with the event trigger and +0.36 [+0.03, +0.70] pp with the periodic trigger. The two values are close, and the protocol did not test their difference.

6.5. Scalability to 40 UAVs

With 40 UAVs and 280 tasks, 90 held-out missions were run under mid disturbances in 4 configurations of latency level and latency factor. All five methods reached between 68.82% and 69.23%. AnytimeSwarm-R reached 69.16% with 127 ms per call. Its pooled difference was +0.02 [−0.26, +0.30] pp to ALNS with 180 ms, +0.34 [+0.05, +0.62] pp to AnytimeSwarm and −0.07 [−0.30, +0.16] pp to the stagnation rule. After Holm correction over the four comparisons of the pre-registered analysis, none of them is significant ($p_{\text{Holm}} = 0.086$ for AnytimeSwarm), so the fifth hypothesis is not supported either. AnytimeSwarm-R again needed less computation than AnytimeSwarm (−9.0 [−11.7, −6.2] ms per call). The stagnation rule stopped earliest, after 100 ms, and was significantly better than AnytimeSwarm-R in 1 of the 4 configurations, which had no latency and $\kappa_B = 50$. On this scale, the rules used between 100 and 250 ms per call, and utility hardly changed within this range of computation. The equal-computation advantage that AnytimeSwarm showed with 40 UAVs under exact execution (Table 4) therefore did not translate into a utility gain under uncertain execution.

6.6. Sensitivity analyses

The sensitivity runs use the held-out missions of the primary matrix without latency and are descriptive. They vary the execution level beyond the primary matrix, the latency factor and the trigger in 8 configurations, and the error model in 4 further configurations.

Under high disturbances, which lie outside the range of the primary matrix but within that of the fitted error models, utility fell to about 48.93% for all methods. The combination of high disturbances and a slow planner ($\kappa_B = 50$) gave the largest advantage of the risk-aware cost in this study. AnytimeSwarm-R served +0.96 [+0.29, +1.66] pp more reward than AnytimeSwarm and used 49 ms instead of 58 ms per call. Its difference to ALNS with 40 ms was +0.14 [−0.53, +0.84] pp and to the stagnation rule +0.43 [−0.33, +1.21] pp, so the gain over AnytimeSwarm brought it to the level of the baselines rather than beyond them. With $\kappa_B = 1$, planning time hardly delays the plan, both delay costs are close to zero and the two rules behave alike. AnytimeSwarm-R then differed from AnytimeSwarm by +0.01 [−0.13, +0.15] pp and −0.5 ms per call, as the reduction to the deterministic cost implies.

Replanning when a UAV deviated by more than 30 s from its planned finish time raised the number of calls from 6.8 to 13.2 per mission and the utility by +0.86 [+0.58, +1.14] pp for AnytimeSwarm-R and +0.86 [+0.54, +1.18] pp for ALNS with 40 ms. Like the periodic trigger, it helped both stopping rules by a similar amount. Under the deviation trigger, the difference of AnytimeSwarm-R was +0.10 [−0.30, +0.52] pp to ALNS with 40 ms and +0.29 [−0.05, +0.65] pp to AnytimeSwarm.

AnytimeSwarm-R was robust to a misspecified error model. With the error model of the low, mid or high level under low and mid disturbances, its difference to AnytimeSwarm ranged between +0.07 [−0.31, +0.47] pp and +0.18 [−0.18, +0.55] pp, and the mean utilities of all versions lay within 0.11 pp. Setting the error model to zero gave −0.13 [−0.32, +0.03] pp. The four misspecification configurations also belong to the primary matrix, so the correctly specified method was run twice on the same missions. The two runs differ only in the measured planning times, because the stopping rule reads a wall clock. They gave identical utility in 73% of the missions, and the mean utility of a method differed by up to 0.12 pp between the runs. Differences of this size between stopping rules should therefore not be read as systematic.

Table 6: Tier C (PX4 SITL, Gazebo, ROS 2): 30 held-out missions per method, mid disturbances, $\kappa_B = 50$. Utility, tasks lost to late plans and skipped at departure per mission, planning time per call; tier B utility on the same rescaled missions for comparison.

Method	Utility C (%)	Lost to delay	Skipped	ms/call	Utility B (%)
ALNS, 250 ms	51.88	0.43	0.57	250.1	50.17
ALNS, 40 ms	54.69	0.03	0.50	40.1	51.72
AnytimeSwarm	53.70	0.03	0.43	47.4	51.90
AnytimeSwarm-R (proposed)	54.18	0.00	0.37	34.7	52.55

6.7. Tier C: PX4 software-in-the-loop with Gazebo

All 120 held-out flights of tier C were completed, that is, 30 missions with four methods each. Two runs did not reach their altitude within 90 s and were repeated as specified. A further 4 runs had a real-time factor between 1.43 and 1.52, below the validity range of 1.6 to 2.2 stated in the protocol. The protocol did not specify how to treat such runs, so they were repeated once with the same seed, like a technical failure, and the original runs were kept. The repeated runs reached the same utility as the original ones, and the comparisons below are identical with either set (−0.51 [−3.24, +2.18] pp and +0.48 [−2.72, +3.46] pp with the original runs). The real-time factor of the analysed runs was 1.95 on average and at least 1.67. A released plan reached all vehicles after 0.152 s of simulation time on average, which is included in the expected latency of AnytimeSwarm-R.

Table 6 summarises the results. AnytimeSwarm-R reached a utility of 54.18%, against 54.69% for ALNS with 40 ms, 53.70% for AnytimeSwarm and 51.88% for the full budget. The paired differences were −0.51 [−3.24, +2.18] pp to ALNS with 40 ms and +0.48 [−2.72, +3.46] pp to AnytimeSwarm, with Holm-adjusted p-values of 1.000 and 1.000. The fourth hypothesis is therefore not supported. With 30 missions, the intervals are much wider than in tier B and cannot resolve differences of the size seen there. Two effects of tier B reappear in tier C. The full budget was again the weakest method (+2.30 [−1.32, +5.71] pp to AnytimeSwarm-R) and the only one that lost a noticeable number of tasks to late plans, 0.43 per mission against at most 0.03 for the others. AnytimeSwarm-R again computed least among the adaptive rules, with 34.7 ms per call against 47.4 ms for AnytimeSwarm, a reduction of 27% (−12.7 [−18.8, −7.2] ms).

The same 30 missions were also run in tier B with the rescaled parameters of tier C. Tier B predicted the outcome of a flight well. The utility of a run in tier C correlated with that of the same run in tier B with $r = 0.84$ (120 runs), and the mean utility per mission with $r = 0.89$ (Fig. 5b). Tier C served slightly more reward on average, 53.61% against 51.58%, so the kinematic model of tier B was, if anything, slightly pessimistic for these missions. The small differences between the methods were not predicted as well. The method means of the two tiers had a Kendall rank correlation of 0.33, and the sign of the per-mission difference between AnytimeSwarm-R and ALNS with 40 ms agreed in 40% of the missions. In tier B, the advantage of AnytimeSwarm-R on these missions was +0.82 [−0.27, +2.16] pp over ALNS with 40 ms and +0.65 [+0.00, +1.50] pp over AnytimeSwarm (Fig. 5a). Both tiers thus agree that the methods differ by at most a few percentage points on this scale, and that the full budget is the weakest search method, while the order of the three short-computation methods is within the noise of both tiers.

7. Discussion and limitations

The held-out study separates three effects that are often studied together. The first is whether to search at all. Under uncertain execution, every ALNS variant served more reward than regret-2 repair in every configuration of the primary matrix, so the benefit of search does not vanish when the plan is executed with errors. The second is how much to search. A long fixed budget lost reward through late plans, and a short fixed budget lost reward on large fleets. The adaptive stopping rules avoided both losses without tuning per scale, and in the primary matrix AnytimeSwarm-R did so with the least computation. The third is when to

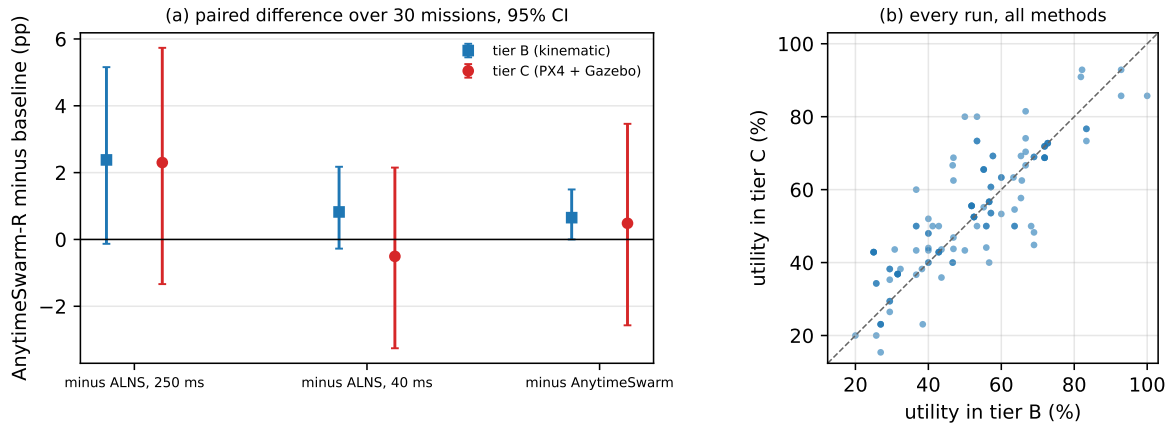


Figure 5: Tier B against tier C on the same 30 held-out missions (three UAVs, mid disturbances, $\kappa_B = 50$): (a) paired difference of AnytimeSwarm-R minus each baseline with 95% bootstrap intervals; (b) utility of every run in tier C against the same run in tier B (dashed: equality).

replan. Periodic replanning raised the utility of every method by more than the choice between the stopping rules did. For a practitioner, this ordering suggests to choose the trigger first and the stopping rule second.

The execution-aware delay cost did not deliver the gain that motivated it. Its pooled advantage over the strongest baselines was a few tenths of a percentage point, and only the comparison with the budget of 40 ms excluded zero before correction. Two explanations are plausible. First, the delay cost only changes the moment at which the search stops. The predicted improvement usually falls quickly, so the deterministic and the risk-aware costs stop the search at similar points, and AnytimeSwarm-R stopped on average 15% earlier than AnytimeSwarm. Second, the remaining finish-time error is corrected by the next replanning call, which follows at the next event or, with the periodic trigger, within 60 s. A risk-aware cost may matter more when replanning is rare or expensive, or when the error model enters the plan itself, for example through stochastic or chance-constrained routing [26, 28], instead of only the decision to stop. The reduction of computation is the more robust benefit. It matters when the planner runs on embedded hardware or shares the processor with perception and control.

The two tiers play different roles. Tier B provides the number of missions that a paired comparison of stopping rules needs, and tier C checks that its conclusions survive an autopilot, a physics engine and a real middleware. Tier B predicted the utility of individual flights closely and reproduced the two robust findings, the loss of the long fixed budget and the lower computation of AnytimeSwarm-R. It did not predict the order of methods whose utilities differ by less than one percentage point, and neither tier can resolve such differences with the number of missions used here. A two-tier study of this kind should therefore state its effect sizes before running, and plan the number of tier C missions from the smallest difference that it is meant to confirm. The per-mission difference between AnytimeSwarm-R and ALNS with 40 ms in tier C had a standard deviation of 7.7 pp. Confirming a difference of half a percentage point with a power of 80% would thus require about 1 800 missions per method, far beyond the 30 missions that a software-in-the-loop campaign of one night allows.

Several limitations bound these conclusions. Tier C uses three UAVs and fifteen tasks per mission, so it validates the decision problem at a small scale and with a limited number of missions. Software-in-the-loop simulation reproduces the autopilot and the vehicle dynamics but not the sensors, the radio link or the weather of a field test. Wind is constant during a mission, and the finish-time error model is linear in the planning horizon and fitted per disturbance level, so the level itself is assumed to be known. The latency factor emulates a slower planning computer instead of measuring one. All experiments ran on a single laptop, and

the measured computation depends on it. Because the stopping rules read a wall clock, repeated runs are not bit-identical, and the mean utility of a method varied by up to 0.12 pp between two runs of the same missions. Finally, the methods share one ALNS and one family of scenarios, and other search methods or mission types may show different gains from adaptive stopping.

8. Conclusion

This paper studied when a multi-UAV replanner should stop computing and publish its plan. AnytimeSwarm stops an anytime ALNS when a learned prediction of the next improvement no longer exceeds a delay cost derived from the slack of the plan, and AnytimeSwarm-R extends this cost with a calibrated model of execution errors and latency. Both rules were evaluated in a pre-registered study with a kinematic tier and a PX4 software-in-the-loop tier on held-out missions.

With exact execution, AnytimeSwarm reached a lower regret than fixed budgets and stagnation stopping at equal computation on every fleet scale. With uncertain execution, the adaptive rules served as much reward as the best fixed budget, which had been matched to them on development data, and avoided the losses of long budgets on slow planners and of short budgets on large fleets without tuning per scale. AnytimeSwarm-R needed 15% less computation than AnytimeSwarm in the kinematic tier and 27% less in the PX4 tier. Its gain in utility was small and not significant, so the execution-aware delay cost is best seen as a way to stop earlier at no loss rather than as a way to serve more reward. Periodic or deviation-triggered replanning raised the utility of every method more than any choice of stopping rule. The kinematic tier predicted the utility of PX4 flights closely, which supports its use for large comparisons, but differences of less than one percentage point between methods remained within the noise of both tiers.

Future work will let the error model shape the routes themselves through chance-constrained planning, choose the replanning trigger and the stopping rule jointly, and run the planner on embedded hardware and in field tests, where the reduction of computation is expected to matter most.

CRediT authorship contribution statement

Cao-Phuc Ha: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. Anh Nguyen Vu Minh: Software, Validation, Writing – review and editing. Phu-Nguyen Le: Supervision, Methodology, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The planner, both simulation tiers, the experiment configurations with their SHA-256 hashes, the pre-registered protocol, all raw results and the scripts that produce every number, table and figure of this paper will be made available in a public repository upon publication.

Acknowledgements

The authors would like to express their sincere appreciation to the Vietnam Aerospace University for supporting this research during the 2026–2027 academic year.

References

[1] B. P. Gerkey, M. J. Matarić, A formal analysis and taxonomy of task allocation in multi-robot systems, The International Journal of Robotics Research 23 (9) (2004) 939–954. doi:10.1177/0278364904045564.

[2] V. Pillac, M. Gendreau, C. Guéret, A. L. Medaglia, A review of dynamic vehicle routing problems, European Journal of Operational Research 225 (1) (2013) 1–11. doi:10.1016/j.ejor.2012.08.015.

[3] H. N. Psaraftis, M. Wen, C. A. Kontovas, Dynamic vehicle routing problems: Three decades and counting, Networks 67 (1) (2016) 3–31. doi:10.1002/net.21628.

[4] S. Ropke, D. Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transportation Science 40 (4) (2006) 455–472. doi:10.1287/trsc.1050.0135.

[5] T. Dean, M. Boddy, An analysis of time-dependent planning, in: Proceedings of the Seventh National Conference on Artificial Intelligence (AAAI-88), AAAI Press, 1988, pp. 49–54.

[6] E. J. Horvitz, Reasoning about beliefs and actions under computational resource constraints, in: Proceedings of the Third Workshop on Uncertainty in Artificial Intelligence, AAAI and Association for Uncertainty in Artificial Intelligence, Seattle, WA, 1987.

[7] S. Russell, E. Wefald, Do the Right Thing: Studies in Limited Rationality, MIT Press, Cambridge, MA, 1991. doi:10.7551/mitpress/2474.001.0001.

[8] S. Zilberstein, Using anytime algorithms in intelligent systems, AI Magazine 17 (3) (1996) 73–83. doi:10.1609/aimag.v17i3.1232.

[9] E. A. Hansen, S. Zilberstein, Monitoring and control of anytime algorithms: A dynamic programming approach, Artificial Intelligence 126 (1–2) (2001) 139–157. doi:10.1016/S0004-3702(00)00068-0.

[10] J. Svegliato, K. H. Wray, S. Zilberstein, Meta-level control of anytime algorithms with online performance prediction, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18), IJCAI Organization, 2018, pp. 1499–1505. doi:10.24963/ijcai.2018/208.

[11] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.

[12] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[13] N. Koenig, A. Howard, Design and use paradigms for Gazebo, an open-source multi-robot simulator, in: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004, pp. 2149–2154. doi:10.1109/IROS.2004.1389727.

[14] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074. doi:10.1126/scirobotics.abm6074.

[15] M. M. Solomon, Algorithms for the vehicle routing and scheduling problems with time window constraints, Operations Research 35 (2) (1987) 254–265. doi:10.1287/opre.35.2.254.

[16] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden, The orienteering problem: A survey, European Journal of Operational Research 209 (1) (2011) 1–10. doi:10.1016/j.ejor.2010.03.045.

[17] M. Gendreau, F. Guertin, J.-Y. Potvin, É. Taillard, Parallel tabu search for real-time vehicle routing and dispatching, Transportation Science 33 (4) (1999) 381–390. doi:10.1287/trsc.33.4.381.

[18] L. Hong, An improved LNS algorithm for real-time vehicle routing problem with time windows, Computers & Operations Research 39 (2) (2012) 151–163. doi:10.1016/j.cor.2011.03.006.

[19] S. Wang, W. Sun, M. Huang, An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows, Computers & Industrial Engineering 191 (2024) 110122. doi:10.1016/j.cie.2024.110122.

[20] D. Sacramento, D. Pisinger, S. Ropke, An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones, Transportation Research Part C: Emerging Technologies 102 (2019) 289–315. doi:10.1016/j.trc.2019.02.018.

[21] H.-L. Choi, L. Brunet, J. P. How, Consensus-based decentralized auctions for robust task allocation, IEEE Transactions on Robotics 25 (4) (2009) 912–926. doi:10.1109/TRO.2009.2022423.

[22] L. Bertuccelli, H.-L. Choi, P. Cho, J. How, Real-time multi-UAV task assignment in dynamic and uncertain environments, in: AIAA Guidance, Navigation, and Control Conference, American Institute of Aeronautics and Astronautics, 2009, AIAA Paper 2009-5776. doi:10.2514/6.2009-5776.

[23] C. Qian, B. Fu, Uncertainty-gated distributed dynamic task allocation for multi-UAV systems under intermittent communication, Drones 10 (10) (2026) 782. doi:10.3390/drones10100782.

[24] S.-Y. Hwang, S. Oh, J.-C. Lee, S. Lee, J.-H. Lee, H.-J. Hwang, Assignment-preserving replanning for dynamic task allocation in multi-UAV systems, Applied Sciences 16 (19) (2026) 9414. doi:10.3390/app16199414.

[25] Y. Shen, T. Hu, Y. Wan, H. Sun, M. Tan, Edge-responsive local-repair particle swarm optimization for dynamic spatiotemporal task allocation of IoT-connected UAV swarms, IEEE Access 14 (2026) 115169–115190. doi:10.1109/ACCESS.2026.3717031.

[26] M. Gendreau, G. Laporte, R. Séguin, Stochastic vehicle routing, European Journal of Operational Research 88 (1) (1996) 3–12. doi:10.1016/0377-2217(95)00050-X.

[27] R. W. Bent, P. Van Hentenryck, Scenario-based planning for partially dynamic vehicle routing with stochastic customers, Operations Research 52 (6) (2004) 977–987. doi:10.1287/opre.1040.0124.

[28] S. S. Ponda, L. B. Johnson, J. P. How, Distributed chance-constrained task allocation for autonomous multi-agent teams, in: Proceedings of the American Control Conference (ACC), 2012, pp. 4528–4533. doi:10.1109/ACC.2012.6315626.

[29] J. Svegliato, P. Sharma, S. Zilberstein, A model-free approach to meta-level control of anytime algorithms, in: Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2020, pp. 11436–11442. doi:10.1109/ICRA40945.2020.9196898.

[30] Y. Sung, L. P. Kaelbling, T. Lozano-Pérez, Learning when to quit: Meta-reasoning for motion planning, in: Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2021, pp. 4692–4699. doi:10.1109/IROS51168.2021.9636864.

[31] M. W. P. Savelsbergh, The vehicle routing problem with time windows: Minimizing route duration, ORSA Journal on Computing 4 (2) (1992) 146–154. doi:10.1287/ijoc.4.2.146.

[32] J.-Y. Potvin, J.-M. Rousseau, A parallel route building algorithm for the vehicle routing and scheduling problem with time windows, European Journal of Operational Research 66 (3) (1993) 331–340. doi:10.1016/0377-2217(93)90221-8.

[33] P. Shaw, Using constraint programming and local search methods to solve vehicle routing problems, in: Principles and Practice of Constraint Programming – CP98, Vol. 1520 of Lecture Notes in Computer Science, Springer, 1998, pp. 417–431. doi:10.1007/3-540-49481-2_30.

[34] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, Science 220 (4598) (1983) 671–680. doi:10.1126/science.220.4598.671.

[35] P. McCullagh, J. A. Nelder, Generalized Linear Models, 2nd Edition, Chapman and Hall, London, 1989. doi:10.1007/978-1-4899-3242-6.

[36] L. Perron, V. Furnon, OR-Tools, https://developers.google.com/optimization, version 9.15 (2026).

[37] C. Voudouris, E. Tsang, Guided local search and its application to the traveling salesman problem, European Journal of Operational Research 113 (2) (1999) 469–499. doi:10.1016/S0377-2217(98)00099-X.

[38] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman and Hall, New York, 1993. doi:10.1007/978-1-4899-4541-9.

[39] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[40] T. Fawcett, An introduction to ROC analysis, Pattern Recognition Letters 27 (8) (2006) 861–874. doi:10.1016/j.patrec.2005.10.010.